OptimalThinkingBench: Evaluating Over and Underthinking in LLMs

- Thinking models can solve complicated tasks by solving the problem step by step. This process costs a lot of tokens. However, such models, like R1 and O3, may "overthink" when they are handling some simple tasks, causing waste in tokens. *OptimalThinkingBench* is proposed to solve such drawbacks.
- An intuitive way to think about this framework is:
  - Let the model do some simple tasks that shouldn't cost a lot of tokens. Find a matric that will combine accuracy and # of token used as a punishment.
  - Let the model do some hard tasks that SHOULD use a lot of tokens to think and measure accuracy.
  - The combined result will reflect the model's ability to select **"when to think a lot"** and balance the # of tokens during the task-solving based on the difficulty of the problem.

This benchmark has 2 subparts: OverthinkingBench and UnderthinkingBench

- OverthinkingBench
  - Data: Based on `OvT-Math` and `OvT-General` datasets. For the former, only Level I and Level II tasks are selected. For the latter, the following two steps are used:
    - Constrained Dataset Generation: Select several domains D(biology, geography, music...) and type of questions T(FRQ, MCQ...) to form a constraint $C = \{D, T\}$. Then, let an LLM $L$to generate $n$question-answer pairs for each C: $L(C) = \{(q_i, a_i)\}_{i=1}^{n}$. The original setting used 72 domains and 4 types of questions: (a) numeric answers, (b) multiple-choice questions (MCQ), (c) one-word or short phrase responses, and (d) open-ended answers.
      - *Advantage: board spectrum of domains; the modular constraints enable systematic ablation studies to understand how overthinking varies with specific domains or answer formats; provides defense against benchmark contamination, since new questions can be generated while maintaining the same properties.*
    - Data filtering: As the questions in this subpart should be easy, we use another LLM $L'$to test the difficulty of the problems given by L. Specifically, we sample $k$answers ($k = 8$ in the paper) from L' and compare with a_i, the ground truth answer provided by L. If all k answers match with a_i, then we keep the C_i. A data point is accepted iff $\forall j \in 1, 2, \ldots, k : L_{judge}(q_i, a_i, y_j) = \text{True}$.
      - *Advantage: Ensures answer correctness, question clarity, and appropriate difficulty.*
  - Evaluation metrics: we track both the **accuracy** and the **token cost**
    - Accuracy: A key point is to use the same judge model L_{judge}in the actual use in the general tasks. In math tasks, programmatic matching (math-verify) is used to ensure exactness. We define correctness as:

      $\text{correctness}_i : L_{judge}(q_i, a_i, y_i) \rightarrow \{0, 1\}$
    - Token cost: This is a punishment term. We use a binary function:

      $\mathbb{I}(\text{Think Tokens} < t)$,

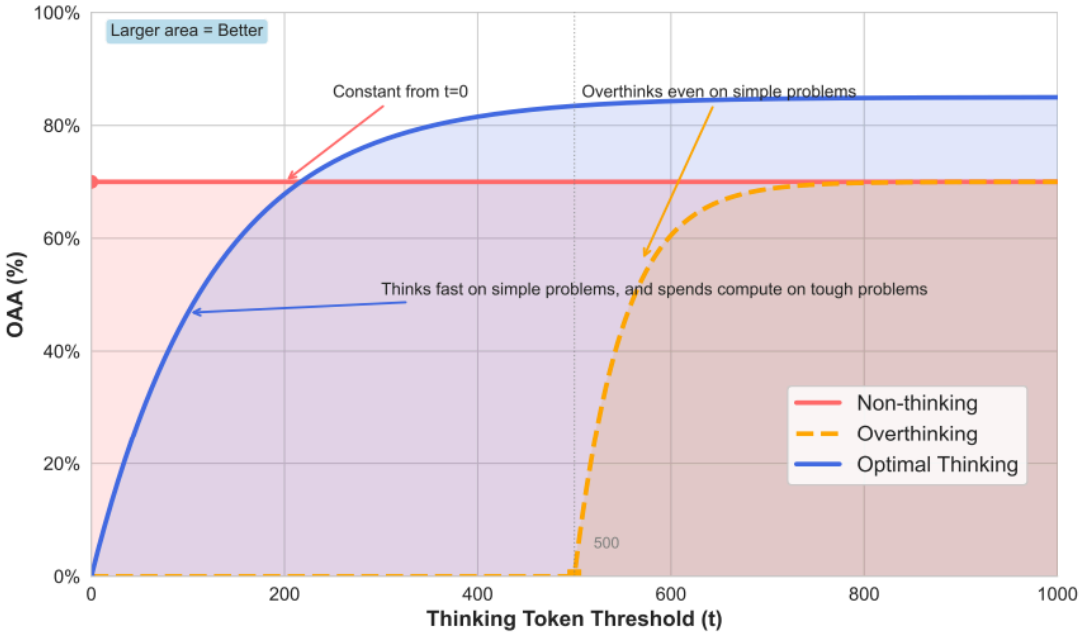
      where $t$is a predefined maximum allowed token cost.

- Combining these two parts together, we have the Overthinking-Adjusted Accuracy (OAA):

  $OAA_t = \frac{1}{n}\sum_{i=1}^{n}(\text{Correctness}_i \cdot \mathbb{I}(\text{Think Tokens}_i < t))$

  - Intuition: even though the answer is correct, if the token cost is too high, OAA of that question is still 0.

- The $t$parameter of OAA is tricky to choose, as a small threshold would cause most thinking models to score 0, while a large threshold would not penalize overthinking.



- Also, the `OAA vs Thinking Token Threshold` plot can reflect the model's ability to control and allocate tokens for thinking. The AUC of the plot is defined as:

  $AUC_{OAA} = \int_0^{t_{max}} \frac{OAA_t}{t_{max}}\, dt \approx \sum_{t=0}^{t_{max}} \frac{OAA_t}{t_{max}}$*(divide by $t_{max}$to normalize the result.)*

  - A non-thinking model's curve will be horizontally straight and keep at a fair level.
  - A thinking model with thinking-token allocation & control ability will be a curve steeply rising from ~0 and has a relatively large AUC_{OAA}.
  - A thinking model without such ability will have a step-like curve: the OAA keeps very low until a relatively large $t$value and increases drastically.

- UnderthinkingBench
  - Data: in this part, we choose hard questions from `Reasoning Gym` along with 4 math benchmarks.
    - Normally, when solving hard questions, a large non-thinking model will perform worse than a small thinking model, as step-by-step thinking plays a more important role than parameter-memorization in solving such problems
    - Taking advantage of this property, we let one small thinking model and one large non-thinking model solve the problems and measure their performance P. Only the questions satisfy

      $P_{\text{small thinking}} - P_{\text{large non-thinking}} > \lambda$($\lambda = 0.1$ in original paper) will be kept. Then, **procedurally generated** the questions and answers to prevent contamination (as the problems generated in this way are all brand new, the testee cannot see them before), easily control the difficulty, and have a ground truth.

    - Evaluation metrics: in this case, we only use accuracy as the metric. As the tasks are procedurally generated, the verification will be done by checking the result of code execution.
- Combined Evaluation Metrics:
    - We combine the AUC_{OAA} and underthinking accuracy together by applying an F1-like pattern: $F_1^{Otb} = 2 \cdot \frac{AUC_{OAA} \cdot ACC_{ut}}{AUC_{OAA} + ACC_{ut}}$
    - In this way, only the model with the ability to control the thinking/non-thinking based on the difficulty of tasks and to properly allocate tokens for thinking (do well on both subparts) can have a high F1 score.
- Experiments

  1. No Model is Optimal

  No current model perfectly balances efficiency and reasoning. OpenAI o3 achieved the highest score (F_1^{otb} 71.1%), while GPT-OSS-120B was the best open-weight model (68.3%).

  2. The Trade-off: Thinking vs. Non-Thinking

    - *Thinking Models* (e.g., R1, Qwen3-Thinking) excel at hard tasks but severely **overthink** on simple queries, often wasting 1300+ tokens on basic facts.
    - *Non-Thinking Models* are efficient on simple tasks but fail complex reasoning. For example, Qwen3's accuracy on hard tasks dropped from **52.4%** (thinking) to **14.9%** (non-thinking).

  3. Mitigation Strategies Struggle

    - *Training Methods* (e.g., reward shaping) reduced overthinking but degraded hard-task accuracy by up to 13%.
    - *Routers* improved performance but still trailed an ideal "Oracle" router by ~15%.
    - *Prompting:* Explicitly telling models "**Don't Overthink**" saved 23% tokens without hurting accuracy, whereas "Think step-by-step" actually worsened performance on simple tasks.

  4. Key Failure Modes

  Models rely on surface cues rather than true difficulty. For instance, adding irrelevant options to a multiple-choice question causes a linear increase in thinking tokens (~42 tokens per option), and models overthink significantly more on questions containing numbers.

- Work Cited
    - Paper
    - Chat History